

Entrega #5 Mini-lenguaje

Jesús Ismael Ramos Recio

Este documento describe la implementación de la Memoria de Ejecución y Máquina Virtual para el Mini-lenguaje. Describe las estructuras de datos usadas y sus principales métodos de acceso. También se explica la indexación de la estructura de memoria con las direcciones virtuales. Al final, se mostrarán casos de prueba donde demuestran el funcionamiento del mini-lenguaje.

Modificaciones realizadas

Antes de empezar con la implementación de la Memoria de Ejecución, se deben mencionar una lista de modificaciones realizadas, ya que para la entrega pasada la generación de cuádruplos de las condiciones, ciclos y funciones no se generaban en el orden correcto. Como soporte para corregir este problema, se utilizó la herramienta "ChatGPT" para ayudar a realizar las modificaciones para generar de forma correcta los cuádruplos. Aquí se listan las modificaciones para lograr la generación correcta de cuádruplos:

1. Creación de reglas auxiliares para p_condicion

Esta fue la implementación del cambio:

```
def p_condicion(p):
    '''
        condicion : IF PARENT_A exp_relacion PARENT_C if_condicion
        estatuto if_fin
                    | IF PARENT_A exp_relacion PARENT_C if_condicion
        estatuto ELSE else_inicio estatuto else_fin
    '''
    result = p[3][0] # resultado de la condicion
    dir_result = p[3][1] # direccion del resultado
    tipo_exp = p[3][2] # tipo del resultado
    if len(p) == 6:
        print(f" If: condicion {result}")
        p[0] = ('if', result, p[5])
    else:
        print(f" If-Else: condicion {result}")
        p[0] = ('if-else', result, p[5], p[7])
```

se agregó la regla 'if_condicion' que su propósito es simplemente evaluar la expresión.

Esta es su implementación:

```
def p_if_condicion(p):
    '''
        if_condicion :
    '''
    # Se evalua la exp_relacion
    _, dir_result, _, = p[-2] # exp_relacion contiene el
    resultado, su direccion y tipo
```

```

    label_fin = traductor.nueva_etiqueta()
    traductor.gen.cuadрупlos.agregar('GOTO', dir_result, None,
label_fin)

    traductor.gen.saltos.push(label_fin)

```

También se creó la regla `p_if_fin`, que sirve para verificar si debe hacer un salto hacia otro procedimiento del programa o no.

También se realizaron estas mismas reglas para el caso de 'else':

```

def p_else_inicio(p):
    '''
    else_inicio :
    '''
    # Se crea la etiqueta para el salto del else
    label_else = traductor.gen.saltos.pop()
    label_fin = traductor.nueva_etiqueta()
    traductor.gen.cuadрупlos.agregar('GOTO', None, None,
label_fin)
    traductor.gen.cuadрупlos.agregar('LABEL', None, None,
label_else)
    traductor.gen.saltos.push(label_fin)

def p_else_fin(p):
    '''
    else_fin :
    '''
    # Fin del cuerpo del else. Se saca el salto de la pila
    label_fin = traductor.gen.saltos.pop()
    traductor.gen.cuadрупlos.agregar('LABEL', None, None,
label_fin)

```

2. Creación de reglas auxiliares para `p_ciclo`

La regla de ciclo también tuvo la misma modificación que la regla de condicion. La única diferencia es que ciclo tiene una regla auxiliar `p_while_inicio`, que sirve para identificar el punto de inicio del ciclo.

```

def p_ciclo(p):
    '''
    ciclo : WHILE while_inicio PARENT_A exp_relacion PARENT_C
while_condicion estatuto while_fin
    '''
    print(f" While: condicion {p[4][0]}")

def p_while_inicio(p):
    '''

```

```

while_inicio :
'''

# Crear etiqueta de inicio
label_inicio = traductor.nueva_etiqueta()
traductor.gen.cuadрупlos.agregar('LABEL', None, None,
label_inicio)
traductor.gen.saltos.push(label_inicio)

def p_while_condicion(p):
'''

while_condicion :
'''

# Se evalua la exp_relacion
_, dir_result, _, = p[-2] # exp_relacion contiene el
resultado, su direccion y tipo
label_fin = traductor.nueva_etiqueta()
traductor.gen.cuadрупlos.agregar('GOTO', dir_result, None,
label_fin)

traductor.gen.saltos.push(label_fin)

def p_while_fin(p):
'''

while_fin :
'''

# Fin del cuerpo de la condicional. Se sacan los saltos de la
pila
label_fin = traductor.gen.saltos.pop()
label_inicio = traductor.gen.saltos.pop()
traductor.gen.cuadрупlos.agregar('GOTO', None, None,
label_inicio)
traductor.gen.cuadрупlos.agregar('LABEL', None, None,
label_fin)

```

Estructuras de Memoria

Esta sección mostrará los diagramas de las estructuras de la memoria virtual. Está organizada en diferentes segmentos según un rango definido de direcciones:

Memoria virtual	
Rango 1 - 1999	Constantes. Solo lectura
Rango 2000-3999	Variables declaradas en el ámbito global
Rango 4000-5999	Variables declaradas dentro de funciones
Rango 6000-7999	Parámetros de funciones
Rango 8000-9999	Variables temporales del compilador
Dirección 8999	Registro de retorno para almacenar temporalmente un valor de retorno de una función

Clase Memoria
<pre>self.memoria = { "global": {2000: 5, 2001: 10}, "cte": {1: 5, 2: 10, ... }, "local": {}, "temp": {}, "param": {} }</pre>
<pre>self.pila_ambitos = [{ "local": {}, "temp": {}, "param": {} }, { "local": {}, "temp": {}, "param": {} }]</pre>

Métodos de Acceso

Se listan los métodos de acceso a las diferentes clases

Clase Memoria:

Clase Memoria	
push_frame()	Nuevo marco para una función
pop_frame()	Eliminar marco actual de la pila
frame_actual()	Retornar el marco actual
obtener_segmento(dir)	Determinar a qué segmento pertenece una dirección

Clase Máquina Virtual	
leer(dir)	Leer un valor según su dirección virtual
escribir(dir, valor)	Escribir un valor a una dirección
ejecutar()	Ejecución de cuádruplos

Test Cases:

Estos son los test cases que se llevaron a cabo para la funcionalidad de la Máquina Virtual:

Prueba 1: lectura y escritura

```
def test_lectura_escritura():
    mem = Memoria()
    vm = MaquinaVirtual([], mem)

    mem.push_frame()

    vm.escribir(2000, 10)
    vm.escribir(8000, 99)

    valor1 = vm.leer(2000)
    valor2 = vm.leer(8000)

    print("Test_lectura_escritura OK")
    print(f"Global (2000) = {valor1}")
    print(f"Temp (8000) = {valor2}")
    print()
```

Salida:

Test_lectura_escritura OK

Global (2000) = 10

Temp (8000) = 99

Prueba 2: Suma y Asignación

```
def test_suma_asignacion():
    mem = Memoria()
    mem.push_frame()

    vm = MaquinaVirtual([], mem)
    vm.escribir(2000, 5)
    vm.escribir(2001, 7)

    cuadрупlos = [
        ('+', 2000, 2001, 8000),
        ('=', 8000, None, 2002)
    ]

    vm.cuadрупlos = cuadрупlos
    vm.ejecutar()

    resultado = vm.leer(2002)
    print("Test_suma_asignacion OK")
    print(f"Resultado = {resultado}")
    print()
```

Salida:

Test_suma_asignacion OK

Resultado = 12

Prueba 3: condición if

```
def test_if():
    mem = Memoria()
    mem.push_frame()

    vm = MaquinaVirtual([], mem)
    vm.escribir(6000, 5)    # x
    vm.escribir(6001, 10)  # y

    cuadрупlos = [
        ('>', 6000, 6001, 8000),    # t1 = 5 > 10 (false)
        ('GOTO', 8000, None, 4),    # si falso salta
        ('=', 6000, None, 2000),    # NO se ejecuta
    ]
```

```

        ('LABEL', None, None, 4),
        ('=', 6001, None, 2000)      # sí se ejecuta
    ]

    vm.cuadрупlos = cuadрупlos
    vm.ejecutar()

    result = vm.leer(2000)
    print("TEST IF OK")
    print(f"Resultado = {result}")
    print()

```

Salida:

TEST IF OK

Resultado = 10

Prueba 4: ciclo while

```

def test_while():
    mem = Memoria()
    mem.push_frame()

    vm = MaquinaVirtual([], mem)
    vm.escribir(6000, 0)    # i = 0

    cuadрупlos = [
        ('LABEL', None, None, 0),

        ('<', 6000, 5, 8000),      # t1 = i < 5
        ('GOTO', 8000, None, 6),

        ('+', 6000, 1, 6000),      # i++

        ('GOTO', None, None, 0),

        ('LABEL', None, None, 6)
    ]

    vm.cuadрупlos = cuadрупlos
    vm.ejecutar()

    result = vm.leer(6000)
    print("TEST WHILE OK")
    print(f"Resultado = {result}")

```

```
print()
```

Salida:

TEST WHILE OK

Resultado = 5